

inframon

혼자 돌리기 — 다른 컴퓨터에서 받기부터 결과까지

이 프로그램은 Python 으로 혼자 돕니다. Claude 나 다른 AI 는 필요 없습니다.
PowerShell 에 한 줄만 붙여넣으면 대시보드까지 열립니다. 그 뒤는 화면에서 누르는
순서대로.

2026-09-09 · <https://github.com/tjddnr8334-sudo/inframon>

가장 쉬운 길 — PowerShell 에 한 줄 붙여넣기

키보드 **Win + X** → **터미널** 을 열고, 아래 한 줄을 그대로 붙여넣고 Enter. 이것으로 0~3 절이 전부 자동입니다.

```
irm https://raw.githubusercontent.com/tjddnr8334-sudo/inframon/main/install.ps1 | iex
```

자동으로 하는 일	걸리는 시간
Python 3.11+ · Git 확인 — 없으면 winget 으로 설치	0~3 분
GitHub 에서 받기 → 내 사용자 폴더 C:\Users\W(이름)\inframon (이미 있으면 최신으로 갱신)	10 초
파이썬 패키지 전부 설치 → SNAP 1.1 GB 무인 설치 → snaphu (WSL 안에)	5~15 분 (처음만)
Earthdata 토큰 — 브라우저가 토큰 페이지를 열면 [Generate Token] → 터미널에 붙여넣고 Enter (계정이 없으면 그냥 Enter 로 건너뛰고 5 절)	30 초
데모 → 진단 → 브라우저에 대시보드	1 분

확인 □ 브라우저에 <http://localhost:8501> 대시보드가 열린다 → **4 절** 로 바로 간다. 고려면 터미널에서 Ctrl + C, 다시 띄우려면 같은 한 줄을 다시 붙여넣는다.

이 한 줄이 안 되는 컴퓨터(인터넷 차단, winget 없음)만 아래 0~3 절을 손으로 합니다.

0. 준비물 — 두 개만 설치 (한 번)

	어디서	주의
Python 3.11 이상	https://www.python.org/downloads/	설치 첫 화면 맨 아래 ' Add python.exe to PATH ' 체크
Git	https://git-scm.com/download/win	전부 기본값으로 '다음'

PowerShell 을 엽니다: 키보드 **Win + X** → **터미널**. 그리고:

```
python --version
git --version
```

확인 □ `Python 3.11.x` 이상, `git version 2.x` 가 찍힌다. 'python 은 인식되지 않는 명령' 이면 Python 재설치(PATH 체크).

1. GitHub 에서 받기

```
cd $HOME
git clone https://github.com/tjddnr8334-sudo/inframon
cd inframon
```

확인 □ `Receiving objects: 100%` 가 찍히고, `cd inframon` 뒤 프롬프트가 `...Winframon>` 로 바뀐다.

2. 전부 설치 (한 줄, 5~10 분)

```
python start.py --full
```

이 한 줄이: 파이썬 확인 → 가상환경(.venv) 생성 → inframon 과 필요한 패키지

전부(torch·scipy·pyproj·rasterio·asf_search·streamlit 등 20 개) 설치 → 데모 실행 → 진단 리포트. 한 번만 하면 됩니다.

확인 □ 마지막 부분에 `판정: ☒ 코어 동작 가능`. 그 위 [의존성] 목록이 전부 ☒.

남는 ⚠ 는 하나뿐이어야 합니다 — **Earthdata 토큰 없음**. 이것은 위성 원본(SLC)을 내려받을 때만 필요하고, 5 번에서 넣습니다. 이미 만들어진 교량 결과를 보는 데는 필요 없습니다.

3. 대시보드 띄우기

```
python start.py --dashboard
```

브라우저가 자동으로 열립니다. 안 열리면 주소창에 **http://localhost:8501** 을 직접 칩니다. 그러면 그 터미널에서 **Ctrl + C**. 다음부터 대시보드만 바로 띄우려면:

```
.venv\Scripts\streamlit run src\inframom\dashboard\app.py
```

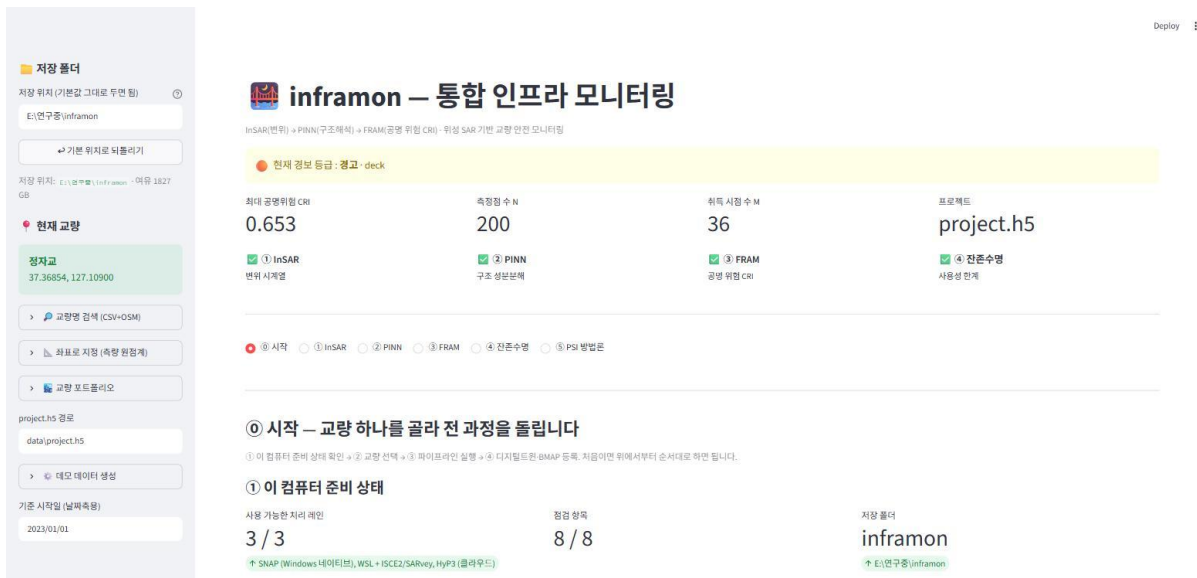


그림 1. 대시보드 — 왼쪽에서 교량을 고르고, 본문 ① 시작에서 전 과정을 돌린다.

확인 □ 그림 1 과 같은 화면. ① 이 컴퓨터 준비 상태 에서 점검 항목이 대부분 ☒ 이고, ☐ 는 Earthdata 하나.

4. 대시보드에서 무엇을 누르나 — 화면 순서대로

대시보드 본문은 위에서 아래로 ① → ② → ③ 순입니다. 스크롤을 내리면서 이 순서로 누릅니다.

4-1. ① 이 컴퓨터 준비 상태 — 보기만

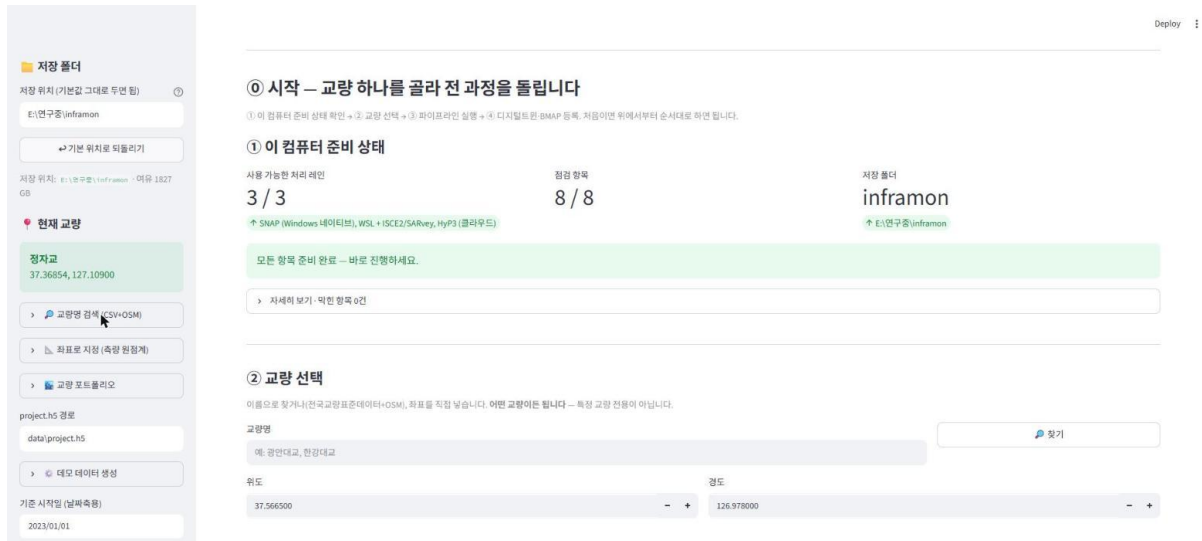


그림 2. ① 준비 상태 — '모든 항목 준비 완료' 면 아무것도 누르지 않고 아래로. 막힌 항목이 있으면 '자세히 보기' 를 펼쳐 설치법을 본다.

확인 □ 모든 항목 준비 완료 — 바로 진행하세요 초록 띠. 막힌 항목 0 건.

4-2. ② 교량 선택 — 이름 또는 좌표

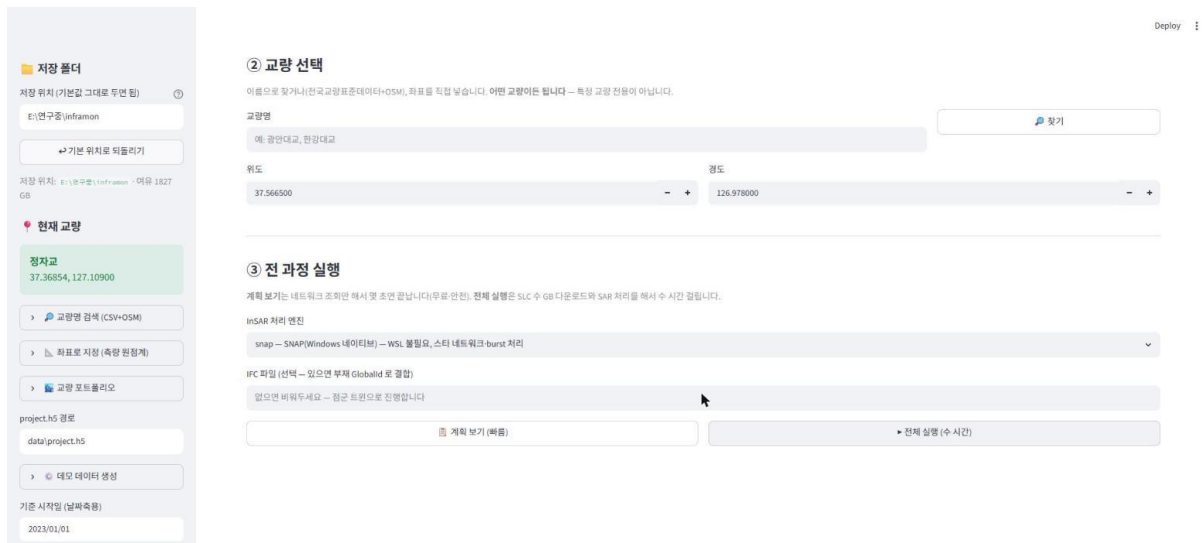


그림 3. ② 교량 선택 — 교량명 입력 후 🔍 찾기, 또는 위도·경도 칸에 직접 입력.

누를 것	무엇을 하나	그다음
교량명 칸에 이름 입력 → 🔍 찾기	전국교량표준데이터+OSM 에서 검색해 위도·경도 칸을 채움	아래 ③ 으로
또는 위도 · 경도 칸에 숫자 입력 후 Tab	예: 마포대교 37.5337 · 126.9366	아래 ③ 으로

확인 □ 위도·경도 칸에 그 교량 좌표가 들어가 있다(기본값 37.5665·126.978 = 서울시청이 아닌지 확인).

4-3. ③ 전 과정 실행 — 버튼 두 개

그림 4. ③ 전 과정 실행 — 왼쪽 계획 보기(수 초, 무료) · 오른쪽 ▶ 전체 실행(수 시간). 누르면 아래 '계획 수립 중...' 이 돈다.

버튼	하는 일	걸리는 시간	필요한 것
계획 보기 (빠름)	어느 궤도에 SLC 가 몇 장 있는지, 교량 제원, 각 단계 계획만	10~30 초	인터넷
▶ 전체 실행 (수 시간)	SLC 다운로드 → SNAP → 언래핑 → PS/DS → PINN → CRI → 트윈 → BMAP 등록	1~3 시간	Earthdata 토큰 · SNAP · snaphu

처음이면 반드시 계획 보기부터 누릅니다. 장면 수가 나오면 전체 실행할 가치가 있는지 판단할 수 있습니다.

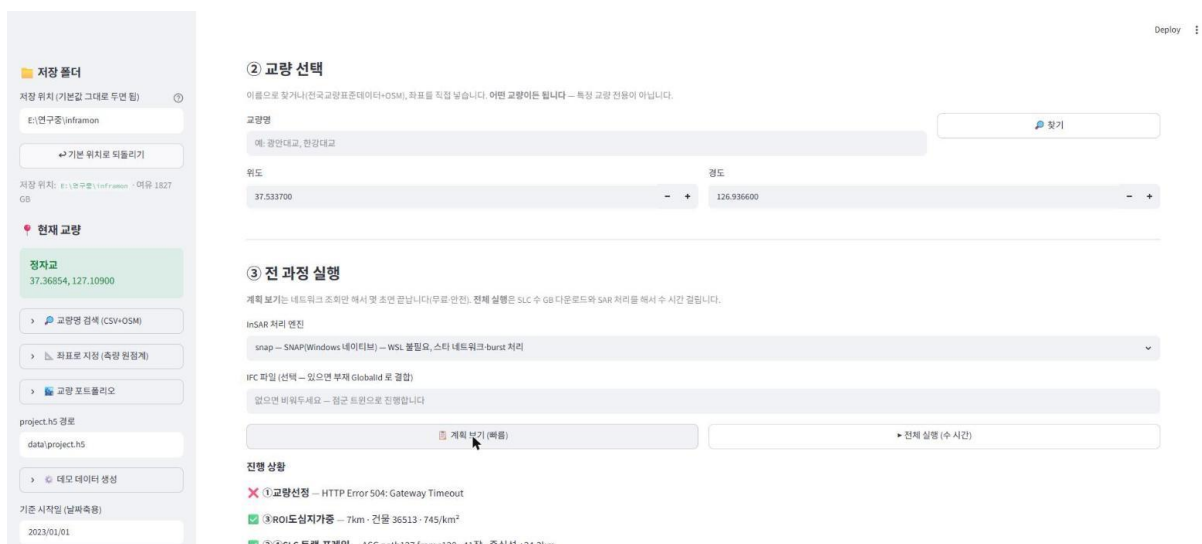


그림 5. 계획 보기 결과 — 진행 상황에 단계별 ☒/X. '②④ SLC·트랙·프레임 ASC path127 · 41 장' 이 이 교량에 쓸 수 있는 위성 장면이다. ①의 X 는 OSM 서버 일시 오류(504) — 다시 누르면 된다.

확인 ☐ 진행 상황에 ②④ SLC·트랙·프레임 ... N 장 이 ☒ 로 나온다. N 이 10 장 미만이면 그 교량은 이 궤도로 어렵다.

4-4. 결과 보기

전체 실행이 끝나면 화면 아래 진행 상황에 ⑫ PINN→FRAM · ⑬ IFC 디지털트윈 · ⑭ BMAP 등록까지 ☒ 가 찍히고, 결과 파일은 왼쪽 저장 위치 폴더 아래 생깁니다. 위 탭 ① InSAR · ② PINN · ③ FRAM 을 눌러 화면에서도 봅니다.

확인 ☐ 진행 상황 마지막 줄까지 ☒. 폴더에 twin.viewer.html 이 생겼다 → 더블클릭.

4-5. 왼쪽 사이드바 — 보조 기능

펼침 항목	언제
교량명 검색 (CSV+OSM)	본문 ② 대신 사이드바에서 검색 — 지도에 마커, 마커 클릭으로 선택
좌표로 지정 (측량 원점계)	TM 좌표(EPSCG:5186 등)로 줄 때
교량 포트폴리오	이미 돌린 교량 목록 — 클릭하면 그 교량 결과로 전환
데모 데이터 생성	위성 자료 없이 프로그램 동작만 확인할 때

4-6. 같은 것을 명령 한 줄로 (대시보드 없이)

```
python scripts\demo_4pm.py # 정자교 44 초, 결과 창 4 개 자동 열림
python scripts\bridge_run.py --name 청양교 --lat 36.450655 --lon 126.80732
python scripts\bridge_run.py --batch docs\bridges\batch.json # 여러 교량 한 번에
```

5. 외부 도구 — Earthdata 토큰 · SNAP · snaphu · SLC 폴더 (한 번)

위성 원본(SLC)부터 돌리려면 셋이 필요합니다. 한 줄 설치기가 이미 했다면 건너뛰니다. 빠진 것만 채우는 방법 둘:

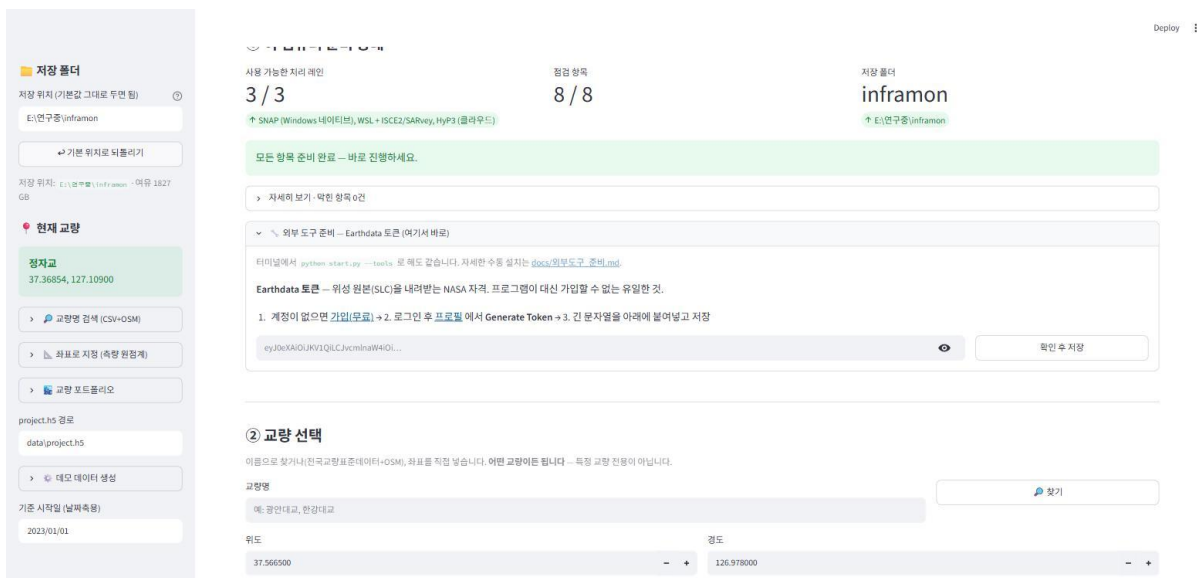


그림 6. 대시보드 ① 준비 상태 아래 '외부 도구 준비' — 없는 것만 나타나고, 토큰은 붙여넣기·SNAP/snaphu 는 버튼.

또는 터미널에서:

```
python start.py --tools
```

도구	프로그램이 하는 일	사람이 할 일
SNAP (ESA)	1.1 GB 내려받아 내 사용자 폴더에 무인 설치, gpt 경로 기록	없음 (기다리기)
snaphu	WSL(Ubuntu) 안에 apt 로 설치. WSL 이 없으면 설치를 걸고 재부팅 안내	관리자 승인 '예', 재부팅 후 같은 명령 한 번 더
Earthdata 토큰	토큰 페이지를 브라우저로 열고, 붙여넣은 토큰을 NASA 서버에 확인한 뒤 저장	가입(무료) → 로그인 → [Generate Token] → 복사 → 터미널에 붙여넣기

확인 □ 마지막 줄 `결과: snap ☒, snaphu ☒, earthdata ☒.

5-1. Earthdata 토큰 — 사람이 하는 3 분

그림 7. <https://urs.earthdata.nasa.gov/users/new> — Username(소문자·숫자·_ 4~30 자) · 비밀번호 12 자+ 대소문자·숫자·특수문자 · 이름 · 메일 · Country · Affiliation → CONTINUE → 메일 인증.

- 로그인 → 프로필(<https://urs.earthdata.nasa.gov/profile>) 위쪽 작은 메뉴 **Generate Token** → 아래 **GENERATE TOKEN** 버튼
- 가려진 토큰 → **Show Token** → `eyJ0eXAiOi...` 전체 복사
- 대시보드 칸(그림 6) 또는 `--tools` 프롬프트에 붙여넣고 Enter → 프로그램이 NASA 서버에 확인한 뒤 저장

토큰은 60 일짜리(최대 2 개). 만료 7 일 전부터 프로그램이 자동 갱신하니 다시 붙여넣을 일은 거의 없습니다. 토큰만 따로: `python -m inframon --earthdata-save <토큰>`. 첫 SLC 다운로드가 401/403 이면 <https://search.asf.alaska.edu> 에 한 번 로그인해 ASF 앱을 승인합니다.

5-2. SNAP · snaphu — 프로그램이 하는 것

	자동	수동 (자동이 안 될 때)
SNAP	ESA 설치기 1.1 GB 내려받아 무인 설치 → AppData\Local\Programs\Wesa-snap. 3~10 분	step.esa.int 에서 Sentinel Toolboxes Windows 설치기 → 전부 Next. 다른 폴더면 INFRAMON_SNAP_GPT 에 gpt.exe 경로
snaphu	WSL 있음 → Ubuntu 안에 apt 설치 1~3 분. WSL 없음 → 관리자 승인 → 재부팅 → 한 번 더	관리자 PowerShell `wsl --install -d Ubuntu` → 재부팅 → `wsl -d Ubuntu -u root -- apt-get install -y snaphu`

각 도구의 '안 될 때' 표(오류 코드별 조치)는 docs/외부도구_준비.md 에 있습니다.

5-3. SLC 보관 폴더 — 원하는 드라이브에

위성 원본은 장당 4~8 GB, 교량 하나에 200~400 GB 까지 갑니다. C: 에 두면 곧 차니 **큰 드라이브**를 고릅니다. 🖱️ 패널 맨 아래 **SLC 보관 폴더**: 드라이브 드롭다운(여유 공간 큰 순) → 폴더(자동 `WSLC`) → **폴더 만들고 저장**. 터미널이면 `python -m inframon --slc-dir E:WSLC` (없으면 만들).

이후 다운로드 는 `E:WSLCW<궤도\_프레임>W` 에 떨어지고, 같은 프레임을 쓰는 다음 교량은 다운로드를 건너웁니다. 이미 받아둔 zip 이 있는 폴더를 지정하면 그대로 인식합니다.

확인 □ 🖱️ 패널에 `현재: E:WSLC (0 장)` 처럼 표시. `--doctor` 의 SLC 보관 폴더 ☒.

그다음 새 교량:

```
python -m inframon --pipeline 37.5337,126.9366 --pipeline-mode plan --out
docs\bridges\마포대교\plan # 계획만(1 분)
python scripts\bridge_run.py --name 마포대교 --lat 37.5337 --lon 126.9366
# 끝까지(1~3 시간)
```

확인 □ 계획: `②④ SLC·트랙·프레임 ASC path127 · 41 장` 처럼 장면 수가 나온다. 끝까지: `⑩ SLC → InSAR` 부터 `⑩` 까지 찍힌다.

6. 결과 보기

파일 (docsWbridgesW<교량>W)	무엇	여는 법
twin.viewer.html	3D 디지털 트윈 — 변위 속도	더블클릭 (인터넷 불필요)
twin_cri.viewer.html	3D 디지털 트윈 — CRI 위험도	더블클릭
brief.png	건기연 형식 4 단 그림 (a)(b)(c)(d)	더블클릭
결과.md	수치 표 · 무엇을 어디서 가져왔나 · 감사 판정 · 못 한 것	메모장·VS Code
<교량>_proxy.ifc	IFC4 교량 모델	Revit · BlenderBIM

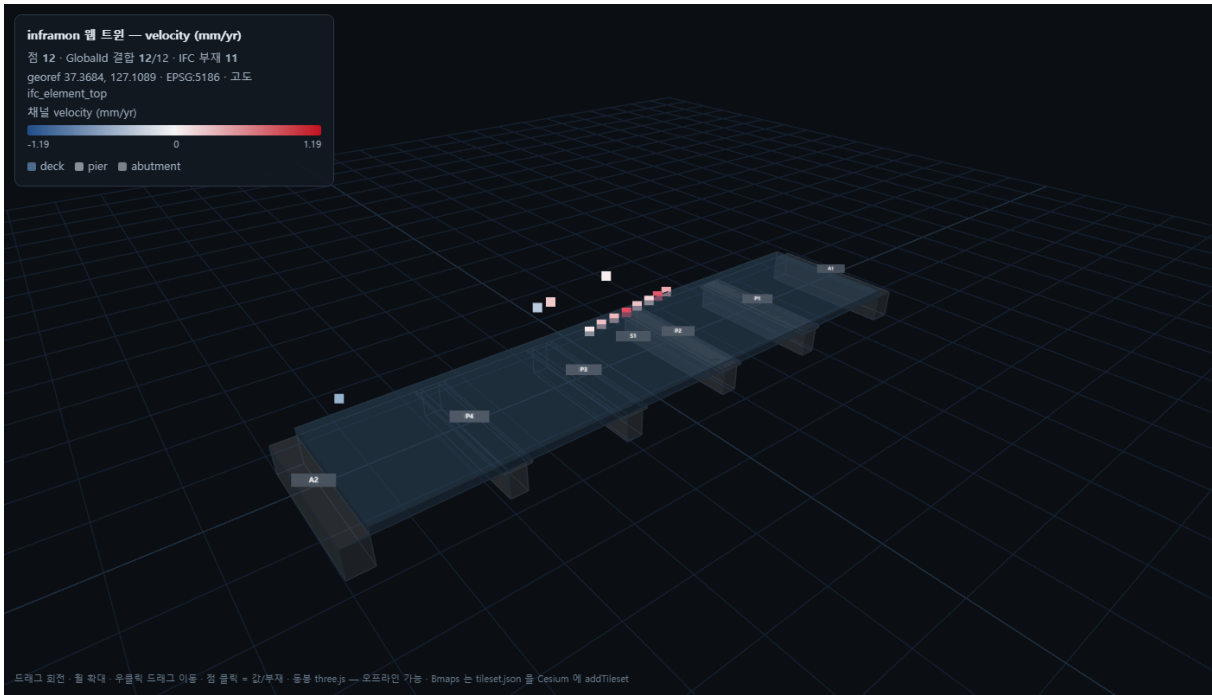


그림 6. twin.viewer.html — IFC 부재(A1~P1~P4~S1) 위 InSAR 점. 드래그 회전, 점 클릭 = 값·부재.

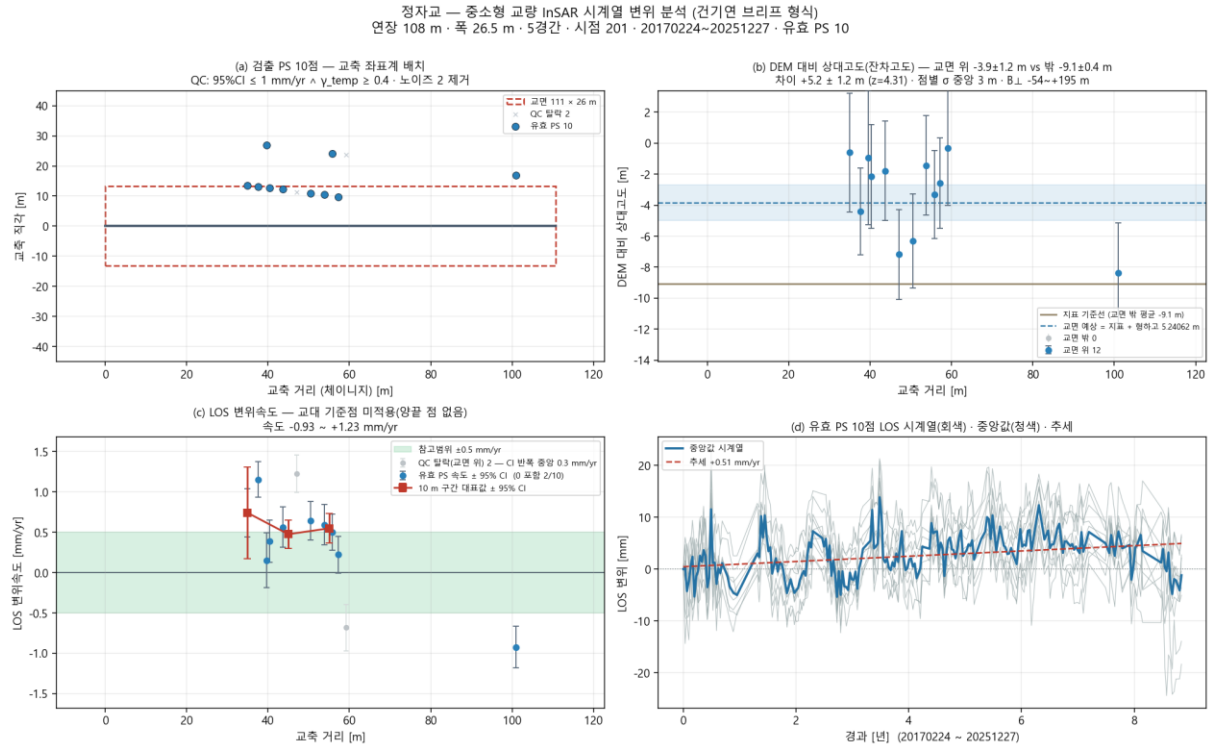


그림 7. brief.png — (a) PS 배치 (b) 잔차고도 (c) 속도 $\pm$ 95% CI (d) 시계열.

6-1. 결과.md 에서 반드시 볼 세 줄

줄	뜻
---	---

감사 보고 가능 / 조건부 / 보고 불가	이 결과를 보고에 써도 되는가를 파일이 스스로 판정. 사유가 뒤에 붙음
① 타·고유진동수는 설계 제원 기반	위성은 상대 변위만 보므로 강성은 관측이 아님 — 감점이 아니라 표기
⚠ 알려진 사고 교량	정자교처럼 사고 이력이 있으면 맨 앞에 적힘. '정상' 을 보고 근거로 쓰지 말 것

7. 막히면

증상	조치
`python` 은 인식되지 않는 명령	Python 재설치, 'Add python.exe to PATH' 체크. PowerShell 새로 열기
`git` 은 인식되지 않는 명령	Git 설치 후 PowerShell 새로 열기
설치가 중간에 멈춤 / 빨간 글씨	인터넷·방화벽 확인 후 같은 명령 다시 (이미 깔린 것은 건너뛸)
8501 이 안 열림	터미널 마지막 줄 주소를 브라우저에 직접. 이미 쓰는 포트면 `--server.port 8502`
'토큰 없음' / 'SNAP gpt 없음' / 'snaphu 없음'	`python start.py --tools` (5 절) — 빠진 것만 채운다
snaphu 에서 'WSL 설치 → 재부팅'	재부팅 후 `python start.py --tools` 한 번 더
OSM 조회 실패(504)	자동 재시도 3 회. 잠시 후 다시. 캐시가 있으면 캐시 사용
실행이 죽음	같은 명령 다시 (받은 위성 자료는 재사용). `결과.md` 의 '예외:' 줄이 사유
'교면 위 점 0'	교량이 작거나 궤도 방향이 불리. 계획(5 번)에서 장면 수·궤도 먼저

8. 전체 그림 — 무엇이 어디서 오나

유저: 교량 좌표 하나

- |
- ├ ① 제원 파트너 실측 CSV → OSM → 전국교량표준데이터
- ├ ② SLC ASF (Earthdata 토큰) ┘ 트랙이 없을 때만
- ├ ③ InSAR SNAP → snaphu 언래핑 ┘ (1~3 시간)
- ├ ④ 점 선택 쉬프트 보정(기하·처리 오프셋) → 교량 위 PS/DS
- ├ ⑤ 잔차고도 점이 지면이 아니라 교면 위임을 고도로 확인
- ├ ⑥ IFC 트윈 실측 제원 → IFC4 → 점을 부재에 결합 → 3D
- ├ ⑦ PINN·CRI 가상센싱(전체 변위장) → 위험도·경보
- ├ ⑧ 브리프 건기연 형식 4 단 그림

└ ㉓ 감사 보고 가능 / 조건부 / 불가 - 파일이 스스로
└ ㉔ 결과.md 전부 기록. 못 한 것은 못 했다고

없는 것은 없다고 적고 넘어갑니다 — 멈추지 않습니다. 어느 교량을 보고에 쓸지는 유저가 정합니다.